

Qwen3-8B, Multi-Needle Counting, and What to Test Next

A synthesis of the current NIAH repo evidence, long-context literature, counting papers, and prompt-sensitivity work

Prepared: 2026-06-17 07:19 UTC

Scope note: this report does not run new experiments. It interprets your README, plan-v7, and main-findings files together with external papers and blogs. Claims about your model behavior are therefore limited to the runs summarized in main-findings-061626.md.

1. Executive takeaways

- The disappointing result is scientifically useful. In your current match_count setup, Qwen3-8B is reliable for counts 1-3 but becomes unstable once the number of inserted city-score records rises above about 3-4. This persists even when the haystack is shortened from 1000 to 500 or 200 tokens [U3].
- This looks less like a pure long-context retrieval failure and more like a cardinality-tracking, set-maintenance, or readout/routing failure. If context length were the only bottleneck, shortening the haystack should have broadly recovered counts 7-10; it did not [U3].
- Small and mid-size recent open models such as Qwen3-8B can count reliably in narrow regimes: small counts, uniform templates, clean delimiters, clear output schema, and favorable prompt formatting. I would not treat unscaffolded multi-needle counting above 3-5 items as reliable without additional prompting, decomposition, or external verification.
- Prompt sensitivity is not an anomaly. It is consistent with prompt-sensitivity literature and with recent work arguing that underspecified prompts often shift the final readout layer more than the internal representation. In your runs, the vanilla prompt is much stronger than the easier prompt, which should be treated as an experimental variable rather than an annoyance [U3].
- The counterfactual needle-span steering result is promising but not yet a robust repair. It gives small-to-moderate gains for selected layer/needle/beta settings, but some settings damage easy examples, and Dataset 9 is not rescued in the harder count sweep [U3].

2. What your current evidence says

The current project is well positioned for this question because it has moved beyond vanilla single-needle retrieval. The counting-feature workflow now uses sentence-level insertion, a single city-score template, vanilla prompting, counterfactual count directions, needle-last feature extraction, and needle-span steering. The README also records that the notebook can generate held-out-style steering-evaluation datasets for multiple gold counts and saves detailed predictions and summaries [U1].

2.1 Baseline counting ladder

The clearest evidence is the count-1-to-count-10 sweep. In the 1000-token run, Qwen3-8B is perfect for counts 1-3, mostly correct for count 4, and unreliable after that. Shorter 500-token and 200-token haystacks improve some mid-count cases, but do not yield reliable counting. Count 9 remains 0.0 across all three context lengths [U3].

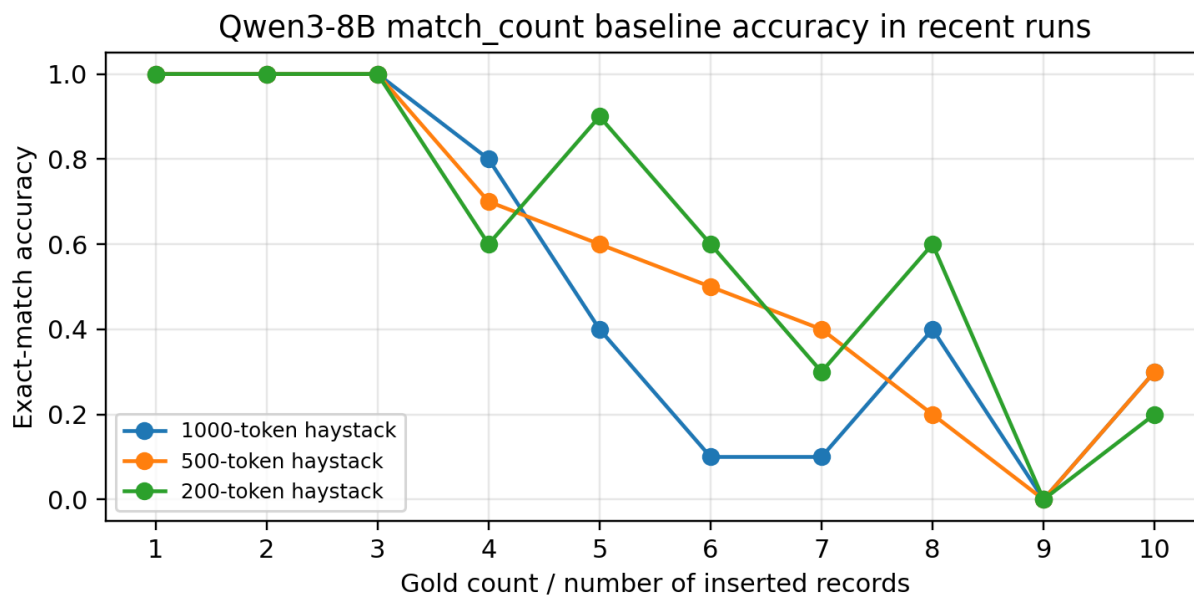


Figure 1. Baseline exact-match accuracy from the recent count-1-to-count-10 evaluations summarized in main-findings-061626.md.

Gold count	1000-token run	500-token run	200-token run
1	1.0	1.0	1.0
2	1.0	1.0	1.0
3	1.0	1.0	1.0
4	0.8	0.7	0.6
5	0.4	0.6	0.9
6	0.1	0.5	0.6
7	0.1	0.4	0.3
8	0.4	0.2	0.6
9	0.0	0.0	0.0
10	0.3	0.3	0.2

The 200-token run has an additional caveat: for high counts, sentence-delimiter replacement insertion can produce dense back-to-back records, which may change the error mode. The average count for Dataset 9 and Dataset 10 overcounted strongly in that run [U3].

2.2 Prompt formatting is a first-order variable

Your post-template runs show a large gap between prompt styles. The easier prompt gave 50/100 and 133/200 baseline accuracy in two runs, while vanilla gave 96/100 and 194/200 in comparable post-template runs [U3]. That is too large to treat as random noise. It means every causal or representation claim should either fix one prompt format or report a prompt-robust interval.

Prompt style / method	Examples	Baseline accuracy
easier, ridge	100	50/100 = 0.50
easier, counterfactual	200	133/200 = 0.665
vanilla, ridge	100	96/100 = 0.96
vanilla, counterfactual	200	194/200 = 0.97

2.3 Steering is informative, not yet a fix

Ridge vectors are useful as probes but have been inert as steering directions in the clean vanilla runs. Counterfactual vectors are more promising: in selected failed examples, counterfactual needle-span steering rescued many interventions, and in held-out eval it improved some count-4 and count-5 settings. But the larger count-5-to-count-10 sweep shows only modest fixed-setting gains, with the best fair setting improving average accuracy from 0.217 to 0.317 [U3].

3. External literature: why this pattern is plausible

3.1 Long context is not the same as useful context

Qwen3-8B has a strong model-card context claim: 32,768 tokens natively and 131,072 tokens with YaRN [E1]. But advertised context length and useful context use are different concepts. RULER was created precisely because vanilla NIAH is too superficial: it adds diverse needles, multi-hop tracing, aggregation, and question answering beyond simple retrieval [E2]. The RULER repository reports Qwen3-8B aggregate scores of 96.3 at 4K, 96.0 at 8K, 91.8 at 16K, 91.2 at 32K, 82.1 at 64K, and 77.4 at 128K [E3]. These are strong aggregate numbers, but they are not a guarantee that Qwen3-8B can enumerate and count many inserted records in your custom prompt.

The Lost in the Middle paper also warns that long-context models often do not use all positions equally: performance is typically strongest near the beginning or end and weaker in the middle [E4]. Your shorter-context failure suggests that lost-in-the-middle is probably not the only issue here, but position and spacing should still be part of the experimental grid.

Online long-context discussions point in the same direction. AI21 argues that NIAH captures something useful but is insufficient for real applications and highlights RULER-style tasks as closer to long-context use [B1]. LangChain's multi-needle blog reports that retrieval degrades as the number of needles increases and that reasoning over retrieved needles is harder than retrieval alone [B2].

3.2 Counting stresses transformers in a way retrieval does not

Several papers treat counting as a core limitation rather than just another retrieval task. When Can Transformers Count to n ? argues that a size-limited transformer is unlikely to implement unbounded counting and empirically observes phase-transition-like failures [E5]. Counting Ability of LLMs and Impact of Tokenization emphasizes that transformer computation is constant-depth and that counting requires reasoning depth that grows with input length; it also stresses that tokenization can undermine counting [E6]. Language Models Need Inductive Biases to Count Inductively finds that Transformers rely heavily on positional embeddings to count out-of-domain, while recurrent architectures count more naturally [E7].

The most directly relevant new result is The Right Answer, the Wrong Direction, which studies Pythia, Qwen3, and Mistral models from 0.4B to 14B. It finds that linear probes can recover counts from intermediate layers with near-perfect accuracy, but the internal count directions are nearly orthogonal to the output-head digit rows, producing a readout bottleneck [E8]. This maps neatly onto your observation that probes and counterfactual directions can contain useful information while generation still miscounts.

A second recent mechanistic paper proposes a System-2-like strategy for large-scale counting: decompose a large count into smaller sub-counts, store latent counts at final item representations, transfer them through attention heads, and aggregate later [E9]. This suggests a concrete direction for your repo: compare direct count generation with explicit enumeration, sub-counting, and chunked aggregation prompts.

3.3 Multi-evidence retrieval makes even strong models brittle

Counting-Stars, NoLiMa, Sequential-NIAH, and related multi-needle benchmarks all point to the same broad lesson: increasing the number of relevant evidence pieces and the need to reason over them exposes failures that simple NIAH misses [E10, E11, E12]. Therefore, Qwen3-8B failing above three city-score records in your setup is not merely a bad benchmark result. It is a useful probe of a known weak region: maintaining a stable set of multiple retrieved records and

mapping that set to a discrete cardinality.

3.4 Prompt sensitivity should be expected and measured

Prompt sensitivity has a substantial literature. PromptRobust/PromptBench shows that small adversarial prompt variations can affect outcomes [E13]. POSIX proposes a prompt-sensitivity metric and finds that increasing parameter count or instruction tuning does not necessarily reduce sensitivity, while one or a few exemplars often reduce it [E14]. Sclar et al. show large performance swings from meaning-preserving prompt formatting changes in open-source models [E15]. A 2026 prompt-underspecification paper argues that much prompt sensitivity comes from weakly constrained prompts and may appear mostly in final layers rather than internal representations [E16].

For your project, this means the vanilla-vs-easier gap is not a nuisance to hide. It is a mechanism-adjacent phenomenon: Qwen3-8B may encode the records but route or read them out differently depending on the instruction/context/query order. This is exactly the kind of effect your hidden-state and causal analyses can study.

4. Is reliable counting possible for small recent models?

My answer is conditional. If by reliable you mean exact JSON accuracy above 95 percent across counts 1-10, multiple prompt formats, randomized positions, and 200-1000+ token haystacks, the current evidence says no for unscaffolded Qwen3-8B in your `match_count` task. It is reliable for counts 1-3, near the edge at count 4, and unstable afterward [U3].

If by reliable you allow scaffolding, then yes, it may be possible in a restricted engineering sense. The model may count better when forced to enumerate records first, when records are chunked and sub-counted, when examples specify the output format, when the prompt uses a model-preferred chat template, or when retrieval/extraction is separated from arithmetic. But that would be a different claim: not "Qwen3-8B internally counts robustly from long context," but "a pipeline using Qwen3-8B can be made reliable by decomposing the task."

The most interesting scientific hypothesis is that Qwen3-8B has partial count information in hidden states but fails to reliably read it into the final digit/JSON token stream. Your counterfactual directions, ridge-vs-steering mismatch, and prompt sensitivity all fit that hypothesis. The next stage should test it directly.

5. Recommended next-step analyses

5.1 Separate retrieval, enumeration, and counting

- 1 List-then-count baseline: ask the model to return a JSON array of all cities that received a score, then count externally. Compare list recall to direct count accuracy. If list recall is high but count is wrong, the bottleneck is aggregation/readout. If list recall is low, the bottleneck is retrieval or item selection.
- 2 Per-record query baseline: for each generated city, ask whether it received a score and what the score was. This estimates item-level recall independent of cardinality generation.
- 3 Two-stage self-prompt: first produce a numbered list of records, then produce the count. Evaluate both the intermediate list and final JSON. This tests the System-2/decomposition hypothesis.
- 4 Candidate logprob baseline: teacher-force candidate answers `{"count":0}` through `{"count":20}`. If the true count has a high logprob rank but generation chooses another count, the problem is generation/readout. If the true count has low rank, the problem is upstream representation or aggregation.

5.2 Treat prompt style as a measured variable

Run a fixed dataset under a prompt grid, not just the currently preferred prompt. At minimum include: vanilla, easier, RULER-like answer prefix, JSON-only minimal, one-shot vanilla, enumerate-then-count, and context-before-query vs query-before-context. Report median, best, worst, and prompt-spread, not only the best prompt. This directly operationalizes the prompt-sensitivity issue and prevents overfitting the mechanism to one format.

5.3 Test the readout-bottleneck hypothesis

- 1 Train count probes at needle-last, every needle span, query span, final prompt token, and generated answer-prefix token across layers 8-32. Compare probe accuracy/R2 to generation accuracy.
- 2 Compute cosine similarity between count directions and output-head rows for digit tokens 0-20 or JSON count-token continuations. A near-orthogonal relation would support the readout-bottleneck story.
- 3 Run logit-lens/direct-logit-attribution for candidate count tokens. Ask where the correct count becomes linearly visible and whether it ever becomes logit-aligned.
- 4 Patch clean count-k activations into corrupted count-(k-1) prompts and reverse-patch corrupted activations into clean prompts. Sweep needle spans, query tokens, final prompt tokens, K/V cache entries, and selected attention-head outputs.

5.4 Diagnose item position and crowding

Vary count, spacing, and placement independently. The key grid is count 1-10 or 1-15; haystack length 200, 500, 1000, 2000; placement as evenly spaced, random separated, random sentence-level, and deliberately back-to-back. For each condition, save item-level recall, direct count accuracy, predicted-count histograms, and attention mass to each needle. This will distinguish lost-in-the-middle, item crowding, and pure cardinality limits.

5.5 Improve steering evaluation without overclaiming

Keep counterfactual needle-span steering, but present it as a target-sensitive intervention. The next version should clear reusable caches and regenerate no-duplicate datasets, increase held-out examples per count, and evaluate policies rather than one fixed beta. Candidate policies include count-dependent layer selection, target-later-needles for undercounting, and adaptive beta chosen from teacher-forced count-logprob margins. Always report damage on easy counts 1-3 alongside rescue on hard counts.

5.6 Add small-model scaling and RULER compatibility

Add a compact scaling panel: Qwen3-4B, Qwen3-8B, Qwen3-14B or 32B if available, Llama-3.1-8B, and one frontier/API model if budget permits. Also generate a RULER-compatible niah_multivalue or niah_multikey-style dataset with controlled number of values. This will tell you whether your failure is Qwen3-8B-specific, small-model-specific, or a general multi-value counting issue.

6. A concrete near-term experiment matrix

Question	Minimal experiment	Interpretation
Can the model retrieve all records?	Ask for JSON array of all city records for counts 1-10.	Low list recall means retrieval/item-selection failure. High list recall plus wrong count means aggregation/readout failure.
Is the correct count in the logits?	Teacher-force candidate count answers 0-20.	True count high-ranked but not generated points to decoding/readout; low-ranked points upstream.
Is count represented internally?	Probe count from layers/tokens; compare to actual answer.	High probe accuracy but low generation accuracy supports readout/routing bottleneck.
Does prompt formatting change representations or only readout?	Run prompt grid and compare hidden probes/logits.	Stable probes but variable logits imply final-layer/readout sensitivity.

Which item is missed?	Per-city recall and attention by needle index.	Systematic misses indicate position/crowding/retrieval bias; random misses indicate cardinality/noise.
Can decomposition solve it?	Enumerate-then-count and chunked sub-count prompts.	Large gains imply a System-2/decomposition route to reliability.

7. Suggested interpretation language for the project

I would describe the current state as follows:

"Qwen3-8B reliably solves the three-record match_count task under the current vanilla, sentence-level, single-template setup, but its unscaffolded exact counting degrades sharply beyond three or four records even at short-to-moderate haystack lengths. This failure is not explained away by context length alone. The evidence is more consistent with a multi-item cardinality-tracking or readout/routing bottleneck. Counterfactual needle-span steering reveals a non-inert count-related direction and can rescue selected examples, but the intervention is not yet robust across counts, layers, target needles, or coefficients."

8. References

- [U1] README.md uploaded 2026-06-17. Current workflow, counting-feature notebook, steering eval, task definitions, and Dynamic NIAH v2 construction.
- [U2] plan-v7.md uploaded 2026-06-17. Current dataset and prompt plan: sentence-level insertion, single city-score template, distinct city constraints, timing/config defaults.
- [U3] main-findings-061626.md uploaded 2026-06-17. Run summaries, prompt-style comparisons, count-1-to-count-10 evaluations, and steering findings.
- [E1] Qwen/Qwen3-8B model card. Hugging Face. Context length and model architecture information.
- [E2] Hsieh et al. 2024. RULER: What is the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.
- [E3] NVIDIA/RULER GitHub README. Qwen3-8B aggregate RULER scores by context length.
- [E4] Liu et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. TACL.
- [E5] Yehudai et al. 2024. When Can Transformers Count to n? arXiv:2407.15160.
- [E6] Zhang, Cao, You. 2024. Counting Ability of Large Language Models and Impact of Tokenization. arXiv:2410.19730.
- [E7] Chang and Bisk. 2024. Language Models Need Inductive Biases to Count Inductively. arXiv:2405.20131.
- [E8] Garcia. 2026. The Right Answer, the Wrong Direction: Why Transformers Fail at Counting and How to Fix It. arXiv:2605.03258. Preprint.
- [E9] Hasani et al. 2026. Mechanistic Interpretability of Large-Scale Counting in LLMs through a System-2 Strategy. arXiv:2601.02989. Preprint.
- [E10] Song, Zheng, Luo. 2024. Counting-Stars: A Multi-evidence, Position-aware, and Scalable Benchmark for Evaluating Long-Context LLMs. arXiv:2403.11802.
- [E11] Modarressi et al. 2025. NoLiMa: Long-Context Evaluation Beyond Literal Matching. arXiv:2502.05167.
- [E12] Yu et al. 2025. Sequential-NIAH: A Needle-In-A-Haystack Benchmark for Extracting Sequential Needles from Long Contexts. arXiv:2504.04713.
- [E13] Zhu et al. 2023. PromptRobust/PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts. arXiv:2306.04528.
- [E14] Chatterjee et al. 2024. POSIX: A Prompt Sensitivity Index For Large Language Models. arXiv:2410.02185.
- [E15] Sclar, Choi, Tsvetkov, Suhr. 2023. Quantifying Language Models Sensitivity to Spurious Features in Prompt Design. arXiv:2310.11324.
- [E16] Pecher et al. 2026. Revisiting Prompt Sensitivity in Large Language Models for Text Classification: The Role of Prompt Underspecification. arXiv:2602.04297. Preprint.
- [B1] AI21 Labs. 2024. Long Context, But Actually. Blog.
- [B2] LangChain. 2024. Multi Needle in a Haystack. Blog.